

8_IO

File Systems

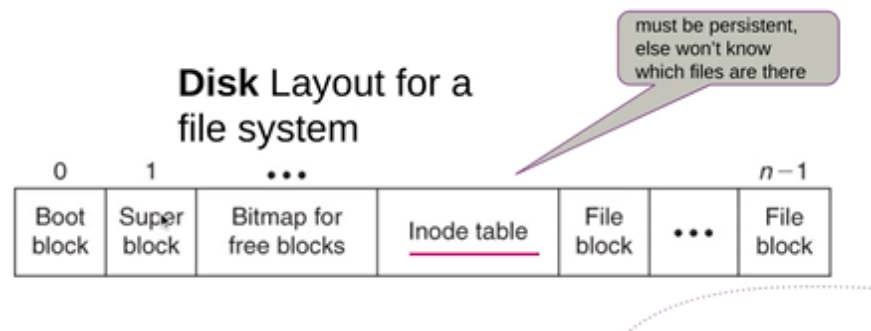
- The file system interface consists of create / delete, open / close, read / write
- "Deep module" - High level, simple API which hides a lot of complexity
- Overview of the file system
 - Path
 - Directory
 - Filename
 - inode - identifies and holds metadata on a file
 - Is a collection of block numbers
 - Also contains some metadata like how big a file is, the type of file etc.
 - To avoid having to carry all the inodes around the file system has an inode table which maps inode ids to the corresponding inodes
 - Can represent which files are in a given directory by having the inode numbers of all the files in the directory in a struct

API: inode

```
structure inode {  
    integer block_numbers[N]; // the number of blocks that constitute the file  
    integer size;             // the size of the file in bytes  
    integer type;             // type of file: regular file, directory, symbolic link  
    integer refcnt;           // count of the number of names for this inode  
    integer userid;           // the user ID that owns this inode  
    integer groupid;          // the group ID that owns this inode  
    integer mode;             // inode's permissions  
    integer atime;            // time of last access (READ, WRITE,...)  
    integer mtime;            // time of last modification  
    integer ctime;            // time of last change of inode  
}
```

- File
 - Each file is a collection of disk blocks / array of bytes

- Block
 - A unit of some size, for example 4KB
 - Often the size of the blocks is the same size of the pages
 - A block device is an array of blocks
 - Can get a specific block based on its id from the block device
 - Hard disk drives are block devices



Symbolic link layer

- A symbolic link is a special type of file that points to another file or directory
 - Basically acts like a shortcut
 - A symbolic link isn't a copy of the file, just a reference to it
- (The link itself has **different permissions** from the target, but accessing it follows the **permissions of the target**)

Symbolic vs Hard Links:

Aspect	Symbolic Link	Hard Link
Points To	Path to another file or directory.	Physical inode of the target file.
Cross File Systems	Can span different file systems.	Cannot cross file systems.
Target Deletion	Becomes a broken link if the target is deleted.	Still retains the data even if the original file is deleted.
Directories	Can point to directories.	Cannot link to directories.
Size	Stores the path name, so its size is small.	Has the same size as the target file.

- Lets us have multiple file names that point to the same data on disk
- The inode struct for the file contain a "ref count" variable that counts how many times this data is referred to
 - If the counter reaches 0 the file is deleted

How about flexible management of files?

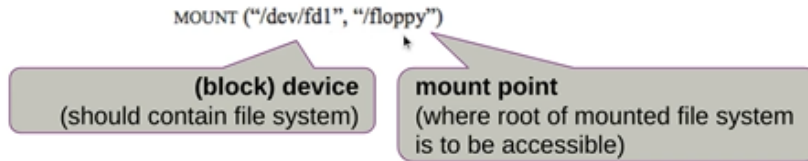
```
LINK (from_name, to_name);  
UNLINK (from_name);
```

```
structure inode{  
    integer block_numbers[N];  
    integer size;  
    integer type;  
    integer refcnt;  
}
```

once no path refers to inode,
it can be garbage collected.

- Can attach new disks to a file system - for example a usb drive

How to attach new disks to a file system?



The file table

- Keeps track of which process is using each file
- (Basically keeps track of open files)
- Keeps track of file name, inode number and cursor, which indicates where in the file the process that has the file open is
 - Cursor is the first byte that will be accessed by the next operation
- The processes keep track of which file they are using in another table called the `fd_table` (file descriptor table)
 - Multiple processes can have the same file open with different cursors
 - Multiple processes can have a file open sharing a cursor (this can for example happen when you fork a process, then the forked process gets a copy of the original process' `fd_table`)

I/O API

in Linux:

```
1 int creat ( const char *pathname, mode_t mode);
2 int remove /* calls `unlink` for files, `rmdir` for directories (refcnt...)
3
4 int link ( const char *oldpath, const char *newpath);
5 int unlink ( const char *pathname);
6
7 int open ( const char *pathname, int flags, ... /* mode_t mode */ );
8 int close ( int fd);
9
10 ssize_t read ( int fd, void buf[.count], size_t count);
11 ssize_t write ( int fd, const void buf[.count], size_t count);
```

- **Create / Delete**
 - To delete a file the references counter is set to 0, then the file system cleans up the file
- **Open / Close**
- **Read / Write**
- **Link / Unlink**
- Other I/O system calls such as `stat, open, lstat, fstat, chmod, chown, utime` etc.

Open

- Mode in the params is the access mode
- Steps:
 - Find inode number of given file
 - If file doesn't exist and we've provided a flag that says that non-existing files should be created when opened then we create the file
 - Get inode corresponding to found inode number
 - Check if we actually have permissions to open the file
 - If successful, we get a file descriptor with which we can access the file
 - Find an empty space in the file descriptor table and insert the file index (this is a row in the file table that we add to represent the open file)

```

procedure OPEN (character string filename, flags, mode) {
    inode_number ← PATH_TO_INODE_NUMBER (filename, wd);
    if inode_number = FAILURE and flags = O_CREATE then { // Create the file?
        inode_number ← CREATE (filename, mode); // Yes, create it.
    } else return FAILURE;
    inode ← INODE_NUMBER_TO_INODE (inode_number);
    if PERMITTED (inode, flags) then { // Does this user have the required permissions?
        file_index ← INSERT (file_table, inode_number);
        fd ← FIND_UNUSED_ENTRY (fd_table); // Yes, find entry in file descriptor table
        fd_table[fd] ← file_index; // Record file index for the file descriptor
        return fd; // Return fd
    } else return FAILURE; // No, return a failure
}

```

Read

```
procedure READ (fd, reference buf, n) {  
    file_index ← fd_table[fd];  
    cursor ← file_table[file_index].cursor;  
    inode ← INODE_NUMBER_TO_INODE (file_table[file_index].inode_number);  
    m = MINIMUM (inode.size - cursor, n);  
    atime of inode ← NOW ();  
    if m = 0 then return END_OF_FILE;  
    for i from 0 to m - 1 do {  
        b ← INODE_NUMBER_TO_BLOCK (i, inode_number);  
        COPY (b, buf, MINIMUM (m - i, BLOCKSIZE));  
        i ← i + MINIMUM (m - i, BLOCKSIZE);  
    }  
    file_table[file_index].cursor ← cursor + m;  
    return m;  
}
```

Linux file system

The virtual file system defines the generic file system interface and data structures:

file, dentry, inode, vfsmount, super_block.

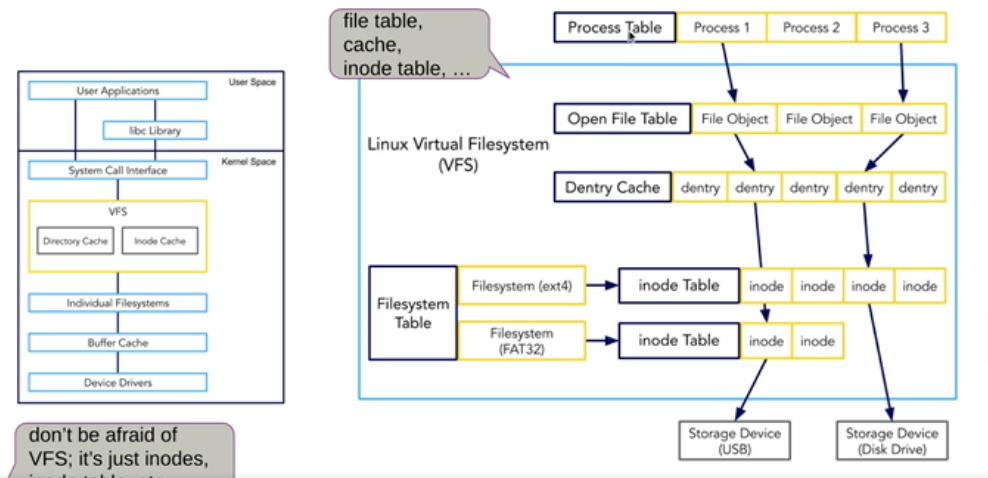
Each specific file system provides a specific implementation:

block-based FS (ext4, btrfs), network FS (NFS, ceph),
stackable FS, pseudo FS (sysfs),
special purpose FS (tmpfs)

Virtual file system (VFS)

Linux VFS

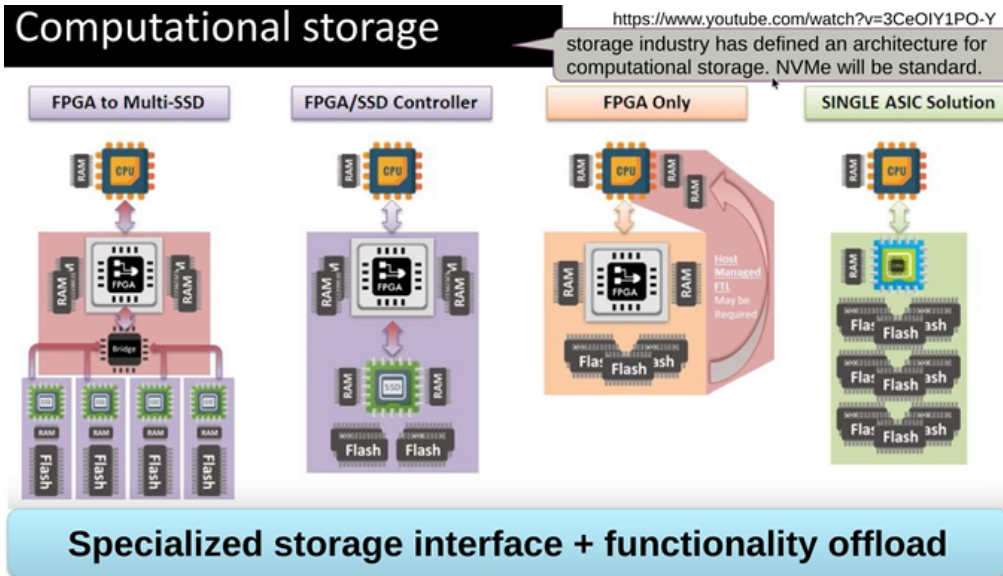
they all respect the VFS setup.



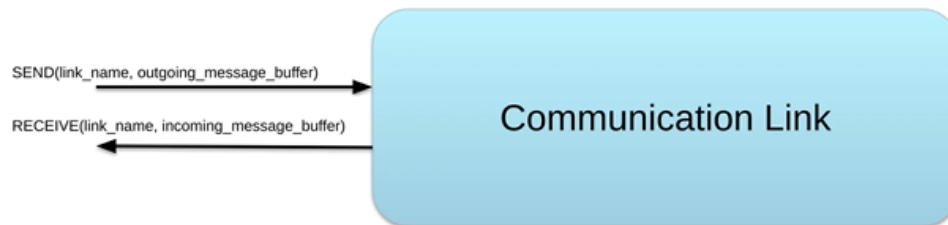
don't be afraid of VFS; it's just inodes, inode table, etc.

Computational storage

- If you're doing simple operations on much data it would be very helpful to be able to avoid having to go through memory and CPU
- In computational storage, processing capabilities are integrated directly into storage devices
 - This enables data processing tasks to be performed closer to where the data is
- Components
 - **Computational Storage Processors (CSPs):** Specialized processing units embedded in storage devices to execute computations locally.
 - **Computational Storage Devices (CSDs):** Storage hardware (e.g., SSDs) equipped with CSPs.
 - **APIs and Frameworks:** Interfaces for programmers to offload tasks to computational storage devices.
- There is an emerging standard for computational storage called NVMe



- Because the CPU isn't reading and writing things in computational storage, the communication has the following structure



In UNIX everything is a file

- In UNIX, everything is a stream of bytes
- In UNIX, the filesystem is used as a universal namespace
- These properties make it so that you can easily implement I/O APIs for any type of file
- For example, in linux you can view the contents of any type of file
- Example of information about the system and special filesystems with which you can interact with the I/O API

Example of pseudo filesystems are:

- › **procfs** (/proc): The proc filesystem contains a hierarchy of special files which can be used to query or control running processes or peek into the kernel internals through standard file entries (mostly text based).
- › **devfs** (/dev or /devices): Devfs presents all devices on the system as a dynamic filesystem namespace. Devfs also manages this namespace and interfaces directly with kernel device drivers to provide intelligent device management – including device entry registration/unregistration.
- › **tmpfs** (/tmp): Temporary filesystem whose content disappear on reboot. Tmpfs is designed for speed and efficiency with features such as dynamic filesystem size as well as memory storage with transparent fallback to swap space.
- › **portalfs** (/p): With the BSD portal filesystem you can attach a server process to the filesystem global namespace. This can be used to provide transparent access to network services through the filesystem. For instance an application could interact with

Sockets

Based on the TCP/IP protocol family

IP (Internet Protocol) :

Provides *basic naming scheme* and unreliable *delivery capability* of packets (datagrams) from *host-to-host*

UDP (Unreliable Datagram Protocol)

Uses IP to provide *unreliable* datagram delivery from *process-to-process*

TCP (Transmission Control Protocol)

Uses IP to provide *reliable* byte streams from *process-to-process* over *connections*

Accessed via a mix of Unix file I/O and functions from the *sockets interface*

1. Hosts are mapped to a set of 32-bit *IP addresses*
130.226.140.95 (2003)
- We are trying to fully replace 32-bit addresses with 64-bits addresses. It has not happened yet (2022)
2. The set of IP addresses is mapped to a set of identifiers called Internet *domain names*
130.226.140.95 is mapped to cos.itu.dk
3. A process on one Internet host can communicate with a process on another Internet host over a *connection*

(3) Internet Connections

Clients and servers communicate by sending streams of bytes over *connections*. Each connection is:

Point-to-point: connects a pair of processes.

Full-duplex: data can flow in both directions at the same time,

Reliable: stream of bytes sent by the source is eventually received by the destination in the same order it was sent.

A *socket* is an endpoint of a connection

Socket address is an `IPAddress:port` pair

A *port* is a 16-bit integer that identifies a process:

Ephemeral port: Assigned automatically by client kernel when client makes a connection request.

Well-known port: Associated with some *service* provided by a server (e.g., port 80 is associated with Web servers)

Well-known Ports and Service

Popular services have permanently assigned *well-known ports* and corresponding *well-known service names*:

echo server: 7/echo

ssh servers: 22/ssh

email server: 25/smtp

Web servers: 80/http

Mappings between well-known ports and service names is contained in the file `/etc/services` on each Linux machine.

Sockets



What is a socket?

To the kernel, a socket is an endpoint of communication

To an application, a socket is a file descriptor that lets the application read/write from/to the network

Remember: All Unix I/O devices, including networks, are modeled as files

Clients and servers communicate with each other by reading from and writing to socket descriptors



The main distinction between regular file I/O and socket I/O is how the application “opens” the socket descriptors

Socket Address Structures

Generic socket address:

For address arguments to **connect**, **bind**, and **accept**

Necessary only because C did not have generic (**void ***) pointers when the sockets interface was designed

For casting convenience, we adopt the Stevens convention:

typedef struct sockaddr SA;

```
struct sockaddr {
    uint16_t sa_family; /* Protocol family */
    char sa_data[14]; /* Address data. */
};
```

Host and Service Conversion: getaddrinfo

```
int getaddrinfo(const char *host,          /* Hostname or address */
               const char *service,       /* Port or service name
*/
               const struct addrinfo *hints, /* Input parameters */
               struct addrinfo **result);  /* Output linked list */

void freeaddrinfo(struct addrinfo *result); /* Free linked list */

const char *gai_strerror(int errcode);    /* Return error msg */
```

Given `host` and `service`, `getaddrinfo` returns `result` that points to a linked list of `addrinfo` structs, each of which points to a corresponding socket address struct, and which contains arguments for the sockets interface functions.

Helper functions:

`freeaddrinfo` frees the entire linked list.

`gai_strerror` converts error code to an error message.

Sockets Interface: socket

Clients and servers use the `socket` function to create a *socket descriptor*:

Example:

```
int socket(int domain, int type, int protocol)
```

```
int clientfd = socket(AF_INET, SOCK_STREAM, 0);
```

Indicates that we are using
32-bit IPV4 addresses

Indicates that the socket
will be the end point of a
connection

Protocol specific! Best practice is to use `getaddrinfo` to generate the parameters automatically, so that code is protocol independent.

Sockets Interface: `bind`

A server uses `bind` to ask the kernel to associate the server's socket address with a socket descriptor:

```
int bind(int sockfd, SA *addr, socklen_t addrlen);
```

The process can read bytes that arrive on the connection whose endpoint is `addr` by reading from descriptor `sockfd`. Similarly, writes to `sockfd` are transferred along connection whose endpoint is `addr`.

Best practice is to use `getaddrinfo` to supply the arguments `addr` and `addrlen`.

Sockets Interface: `listen`

By default, kernel assumes that descriptor from `socket` function is an *active socket* that will be on the client end of a connection. A server calls the `listen` function to tell the kernel that a descriptor will be used by a server rather than a client:

```
int listen(int sockfd, int backlog);
```

Converts `sockfd` from an active socket to a *listening socket* that can accept connection requests from clients.

`backlog` is a hint about the number of outstanding connection requests that the kernel should queue up before starting to refuse requests.

Sockets Interface: accept

Servers wait for connection requests from clients by calling `accept`:

```
int accept(int listenfd, SA *addr, int *addrlen);
```

Waits for connection request to arrive on the connection bound to `listenfd`, then fills in client's socket address in `addr` and size of the socket address in `addrlen`.

Returns a *connected descriptor* that can be used to communicate with the client via Unix I/O routines.

Sockets Interface: connect

A client establishes a connection with a server by calling `connect`:

```
int connect(int clientfd, SA *addr, socklen_t addrlen);
```

Attempts to establish a connection with server at socket address `addr`

If successful, then `clientfd` is now ready for reading and writing.

Resulting connection is characterized by socket pair

(`x:y`, `addr.sin_addr:addr.sin_port`)

`x` is client address

`y` is ephemeral port that uniquely identifies client process on client host

Best practice is to use `getaddrinfo` to supply the arguments `addr` and `addrlen`.

Parallel & Distributed Processing

models:

- client/server
- pipeline (each worker processes data independently)
- boss/worker (one worker process orchestrates)
- peer-to-peer (coordination/consensus algorithm needed)

communication: via. message-passing, following a **protocol**.

example paradigm: **scatter/gather** (example in book); boss scatters array across workers, gathers result from them.